



Sejarah & Arsitektur Web

History of the Web & Web Architecture

Hendrik

CPMK KK1.SIF302.1 · Pertemuan 2 ·
PABW 2026/2027

Prodi Informatika — Universitas Islam Indonesia

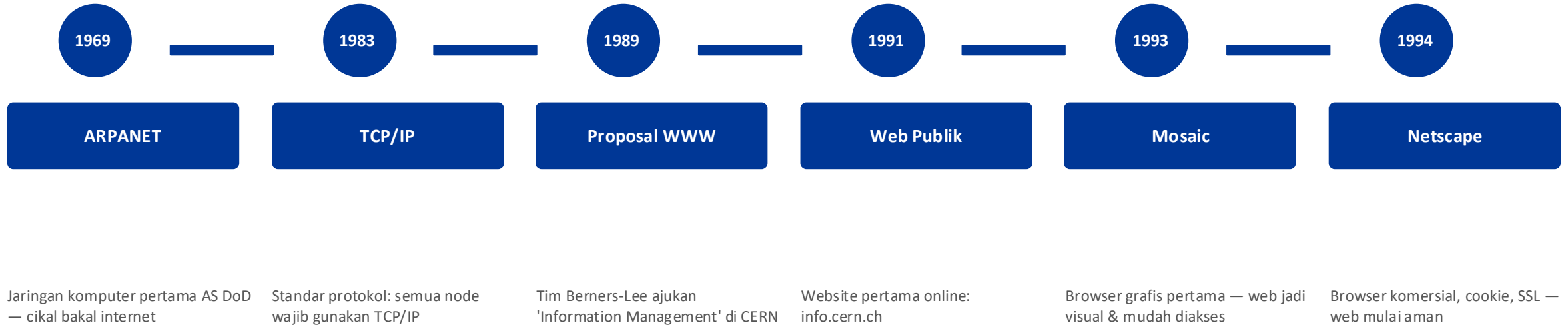
Tujuan Pembelajaran

- 1 Menjelaskan evolusi web dari era 1.0 hingga Web3
- 2 Memahami perjalanan request dari URL ke halaman yang tampil
- 3 Menjelaskan peran browser, server, dan protokol HTTP
- 4 Membedakan server-side vs client-side processing
- 5 Memahami arsitektur modern: MPA, SPA, dan SSR
- 6 Menjelaskan pola desain MVC dan alur kerjanya

Sejarah Web

History of the Web

Dari ARPANET ke World Wide Web



⚠ Web ≠ Internet — Web adalah salah satu layanan yang berjalan di atas infrastruktur internet

Era 1993–2004: Perang Browser & Web Dinamis

Setelah web publik lahir, pertumbuhan eksplosif diikuti kekacauan standar.

1993

Mosaic

Browser grafis pertama. Web jadi visual, bisa diklik, dapat diakses siapapun.

1995

PHP & CGI

Halaman dinamis server-side. PHP, ASP, Perl CGI jadi fondasi web dinamis.

1994

Netscape

Netscape Navigator kuasai ~80% pasar. Cookie, SSL, form interaktif lahir.

1996

IE vs Netscape

Microsoft bundel IE di Windows. Perang browser: kode tidak portabel.

1995

JavaScript

Brendan Eich ciptakan JS dalam 10 hari. Halaman web bisa punya logika.

2000

Dotcom Crash

Gelembung internet pecah. Hanya yang punya nilai nyata yang bertahan.

⚠ Perang browser = kode ditulis dengan hack khusus per browser. Ini mendorong lahirnya standar W3C dan HTML5.

Evolusi Web: Tiga Era

Web 1.0

1991 – 2000-an

Read-Only Web

- Halaman statis (HTML murni)
- Pengguna hanya membaca
- Tidak ada interaksi
- Konten dibuat publisher

Contoh: Wikipedia awal, situs berita statis

Web 2.0

2004 – 2015

Read-Write Web

- Halaman dinamis & interaktif
- User-generated content
- AJAX – update tanpa reload
- Media sosial, blog, wiki

Contoh: Facebook, YouTube, Gmail, Twitter

Web 3.0

2015 – kini

Intelligent Web

- Data terstruktur & semantik
- AI & machine learning
- Real-time & edge computing
- Linked Data & Knowledge Graph

Contoh: Semantic search, personalisasi AI

Web 3.0 vs Web3 — Dua Konsep yang Berbeda

Web 3.0 — Semantic Web

- Digagas Tim Berners-Lee (W3C)
- Data terstruktur & bisa dibaca mesin
- Linked Data & ontologi (RDF, OWL)
- Inferensi otomatis oleh agen AI
- Contoh: DBpedia, Schema.org, Wikidata
- Tujuan: Web yang 'mengerti makna'

VS

Web3 — Decentralized Web

- Konsep berbasis teknologi blockchain
- Aplikasi terdesentralisasi (DApps)
- Kepemilikan aset digital (NFT, token)
- Keuangan terdesentralisasi (DeFi)
- Contoh: Ethereum, IPFS, MetaMask
- Tujuan: Web tanpa otoritas terpusat

⚠ Hati-hati: 'Web 3.0' (Semantic Web) dan 'Web3' (blockchain) adalah dua hal yang sering disalahartikan.

Standar & Tata Kelola Web — Siapa yang Mengatur?

Web tidak dimiliki satu perusahaan. Ada organisasi independen yang menetapkan standar terbuka.

W3C

World Wide Web Consortium

Tim Berners-Lee, 1994

- HTML (historis), CSS, SVG, aksesibilitas (WCAG)
- WebAssembly, Web Components, Linked Data/RDF
- Standar via Working Group — bisa bertahun-tahun

WHATWG

Web Hypertext App Technology WG

Apple, Mozilla, Opera, 2004

- HTML Living Standard — versi HTML di browser nyata
- DOM, Fetch, URL, Storage specification
- Standar 'hidup': diperbarui terus tanpa versi rilis

TC39

Ecma International — TC39

Netscape, Sun, 1997

- Standar ECMAScript = bahasa JavaScript
- Proposal: Stage 0 → Stage 4 sebelum masuk spec
- ES6 (2015) revolusi besar; kini rilis tahunan

IETF

Internet Engineering Task Force

Komunitas internet, 1986

- HTTP/1.1, HTTP/2, HTTP/3 (RFC 9114)
- TLS 1.2, TLS 1.3 — enkripsi web
- DNS, TCP/IP, dan protokol fondasi internet

Web Hari Ini — Landscape 2026

Web telah berevolusi dari dokumen hypertext menjadi platform aplikasi penuh.

PWA

Progressive Web App

- Instal di homescreen tanpa app store
- Mode offline via Service Worker
- Notifikasi push seperti native app
- Twitter, Pinterest, Starbucks gunakan PWA

Jamstack

JS + API + Markup

- Build-time pre-render, serve via CDN
- No server — hanya static files + API
- Framework: Next.js, Nuxt, SvelteKit, Astro
- Sangat cepat, murah, aman by default

Edge

Compute dekat pengguna

- Kode jalan di 200+ lokasi CDN global
- Latensi <50ms ke mayoritas pengguna
- Cloudflare Workers, Vercel Edge, AWS
- Cocok untuk personalisasi & geo-routing

AI Dev

AI sebagai rekan coding

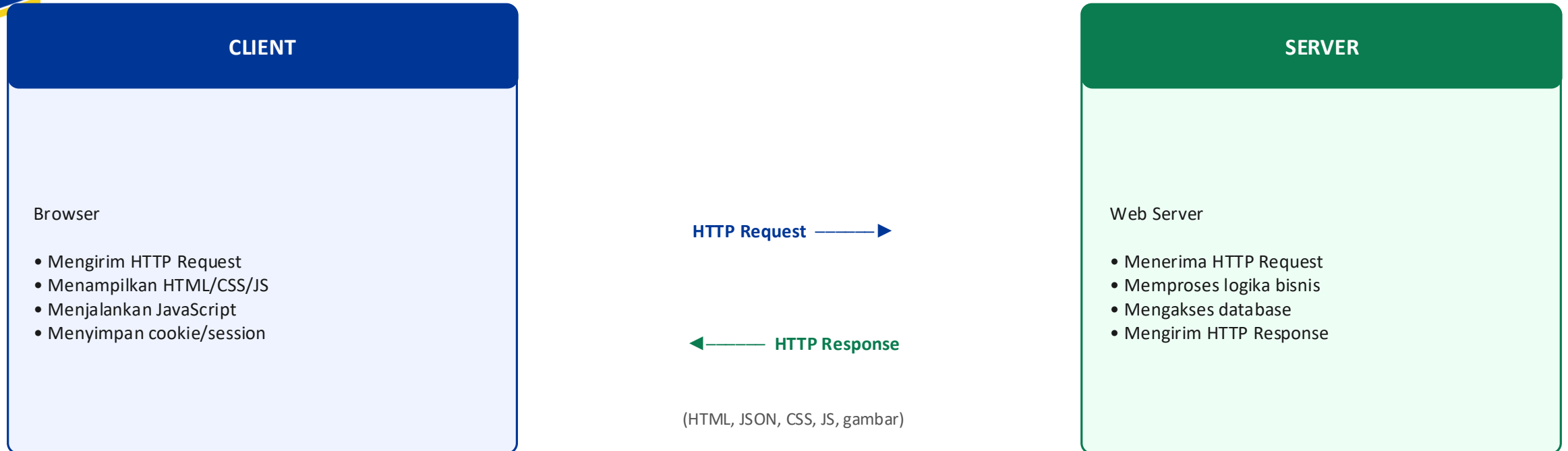
- GitHub Copilot, Claude, Cursor, Bolt
- Generate boilerplate, refactor, debug
- Praktik industri standard 2025–2026
- Tetap butuh engineer yang paham konsep

Core Web Vitals (LCP, CLS, INP) menjadi metrik kualitas standar — Google gunakan sebagai faktor ranking SEO.

Arsitektur Web Dasar

Basic Web Architecture

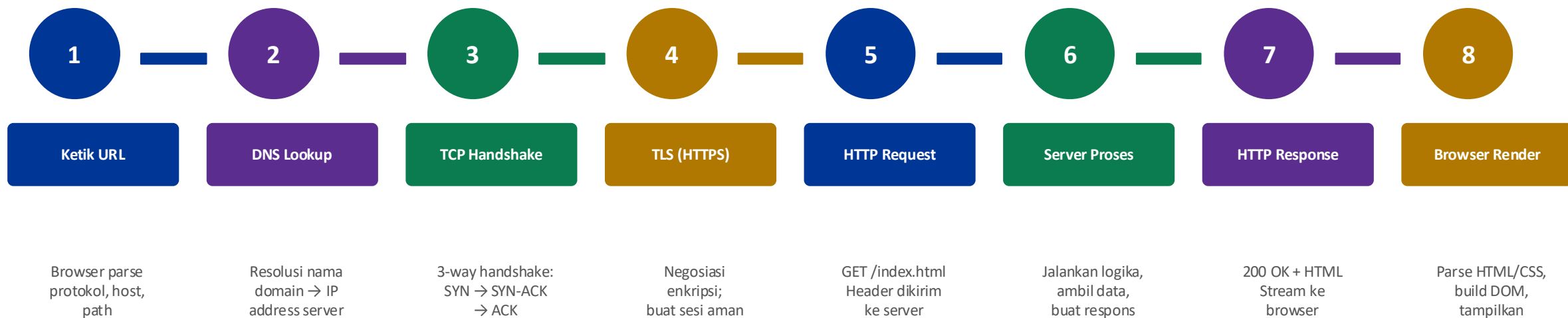
Model Client-Server



Evolusi Arsitektur

2-tier: Client ↔ Web Server | 3-tier: Client ↔ App Server ↔ Database | N-tier: tambah layer (cache, CDN, microservices)

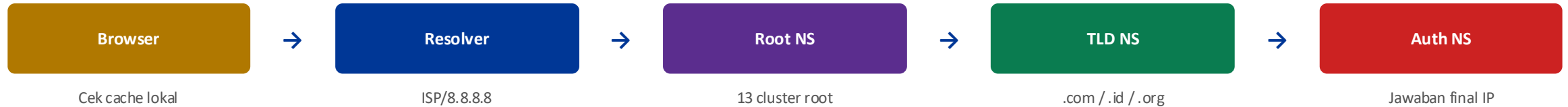
Perjalanan URL ke Halaman — Apa yang Terjadi?



Semua ini terjadi dalam milidetik. Memahami alur ini kunci debugging masalah koneksi, CORS, redirect, dan security.

DNS — Dari Nama Domain ke Alamat IP

Sebelum HTTP request dikirim, browser harus tahu IP address server. DNS yang menyelesaikannya.



Tipe Record DNS yang Penting:

Record	Nama	Fungsi
A	Address	Domain → IPv4 address (103.x.x.x)
AAAA	Addr v6	Domain → IPv6 address
CNAME	Alias	Domain → domain lain (www → @)
MX	Mail	Routing email ke mail server
TXT	Text	Verifikasi domain, SPF, DKIM, dll.

TTL — Time to Live

Seberapa lama hasil DNS di-cache oleh resolver.

TTL rendah (300 dtk)

Perubahan cepat, tapi lebih banyak query DNS
→ Pakai saat mau ganti IP server

TTL tinggi (86400 dtk)

Cache 1 hari — efisien, query sedikit
→ Perubahan butuh waktu lama berlaku

Saat migrasi server

Turunkan TTL dulu beberapa hari sebelum ganti IP
→ Agar transisi mulus di seluruh internet

💡 Cek DNS record domain: `dig uii.ac.id A` atau `nslookup uii.ac.id` di terminal.

Hosting & Deployment — Tiga Konsep yang Berbeda

Domain, sertifikat TLS, dan hosting adalah tiga hal yang terpisah — bisa dari provider berbeda.

Domain

Nama yang diketik di browser.
Didaftarkan ke registrar (Niagahoster, Cloudflare, GoDaddy).
Bayar per tahun. Dihubungkan ke hosting via DNS record A/CNAME.

uii.ac.id, contoh.com

SSL/TLS

Enkripsi koneksi browser ↔ server (HTTPS).
Let's Encrypt: gratis, auto-renew 90 hari.
Sebagian besar hosting modern sudah include otomatis.

Tanda  di browser

Hosting

Tempat file dan aplikasi web disimpan & dijalankan.
Bentuknya bisa bermacam-macam — lihat tabel opsi di bawah.

Opsi lihat tabel →

Opsi Hosting:

Shared Hosting

Server dibagi banyak user. Murah (~Rp50rb/bln).
Kendali terbatas. Blog, landing page

Static / Edge CDN

Deploy HTML/JS ke CDN global. Sangat cepat, ada gratis tier. Vercel, Netlify, CF

VPS / Cloud VM

Server virtual sendiri. Kendali penuh (SSH). Perlu sysadmin. EC2, DigitalOcean

PaaS

Deploy app tanpa urus infrastruktur. Bayar per usage. Railway, Fly.io

Evolusi Arsitektur: dari 2-tier ke N-tier

2-Tier (Basic)

Client (Browser)

↕ HTTP

Web Server

Sederhana. Cocok untuk situs statis.
Skalabilitas terbatas.

3-Tier / N-Tier (Modern)

Client (Browser)

Web Server / CDN

Application Server

Database Server

Separation of concerns. Skalabel, mudah di-maintain.

HTTP & Protokol Web

HTTP & Web Protocols

HTTP — HyperText Transfer Protocol

Berbasis TCP/IP

HTTP berjalan di atas TCP/IP. TCP menjamin paket terkirim dan teratur. HTTP mengatur apa yang dikomunikasikan antara browser dan server.

Request–Response

Model komunikasi dua arah: client mengirim request, server membalas dengan response. Setiap resource diakses lewat satu siklus request-response.

Resource via URI

Setiap resource web (halaman, gambar, API endpoint) diidentifikasi dengan URI/URL. Browser menggunakan URL untuk tahu apa yang harus diminta ke server.

HTTP adalah stateless — server tidak menyimpan informasi antar request. Setiap request berdiri sendiri.

Anatomi HTTP Request & Response

HTTP REQUEST

Request Line

GET /index.html HTTP/2

Headers

Host: example.com
Content-Type: text/html
Accept-Language: id

Empty Line

(pemisah header & body)

Body

Data form, JSON payload
(kosong untuk GET)

HTTP RESPONSE

Status Line

HTTP/2 200 OK

Headers

Content-Type: text/html
Cache-Control: max-age=3600
Set-Cookie: session=abc

Empty Line

(pemisah header & body)

Body

<html>...</html>
atau JSON, gambar, dll.

HTTP Methods — Jenis Aksi pada Resource

Method	Deskripsi	Safe?	Idempotent?	Contoh
GET	Ambil/baca resource	Ya	Ya	GET /users/123
POST	Kirim data / buat resource	Tidak	Tidak	POST /users
PUT	Ganti seluruh resource	Tidak	Ya	PUT /users/123
PATCH	Update sebagian resource	Tidak	Ya	PATCH /users/123
DELETE	Hapus resource	Tidak	Ya	DELETE /users/123



HTTP Status Codes

1xx Info

100	Continue
101	Switching Prot.

2xx Sukses

200	OK
201	Created
204	No Content

3xx Redirect

301	Moved Permanently
302	Found
304	Not Modified

4xx Client Err

400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found

5xx Server Err

500	Internal Srv Err
503	Unavailable

HTTP vs HTTPS — Mengapa HTTPS Wajib?

HTTP (Tidak Aman)

- ✗ Data dikirim sebagai plain text
- ✗ Siapapun di jaringan bisa membaca
- ✗ Tidak ada verifikasi identitas server
- ✗ Rentan terhadap man-in-the-middle attack
- ✗ Browser menampilkan 'Not Secure'
- ✗ Tidak diizinkan untuk situs modern

HTTPS (HTTP + TLS)

- ✓ Data dienkripsi dengan TLS
- ✓ Privasi — tidak terbaca pihak ketiga
- ✓ Integritas — data tidak dapat diubah
- ✓ Autentikasi — server terverifikasi via sertifikat
- ✓ Browser menampilkan ikon gembok (🔒)
- ✓ Diwajibkan sejak ~2017 oleh semua browser besar

TLS (Transport Layer Security) adalah protokol kriptografi yang menjamin privasi, integritas, dan autentikasi di atas TCP.

Evolusi HTTP: dari 1.1 ke HTTP/3

HTTP/1.1

1997

- Persistent connection (keep-alive)
- Request–response sequential
- Head-of-line blocking
- Satu request per koneksi TCP

Masih umum; dibatasi performa

HTTP/2

2015

- Multiplexing — banyak request sekaligus
- Header compression (HPACK)
- Server push (resource sebelum diminta)
- Binary framing (lebih efisien dari teks)

Standar default di web modern

HTTP/3

2022

- Berbasis QUIC (UDP, bukan TCP)
- 0-RTT connection (lebih cepat)
- Tidak ada head-of-line blocking
- Built-in enkripsi TLS 1.3

Digunakan 30%+ web. Masa depan protokol.

HTTP Session State — Stateless & Solusinya

HTTP adalah protokol stateless — server tidak mengingat siapa kita antar request.

Masalah: jika server tidak ingat kita, bagaimana login tetap aktif saat berpindah halaman?

Cookie

File kecil disimpan di browser. Server set via header Set-Cookie. Dikirim otomatis di setiap request ke domain yang sama.

Session

Data disimpan di server, tanda pengenal (session ID) dikirim ke browser sebagai cookie. Lebih aman untuk data sensitif.

URL Parameter

Session ID di URL: /page?session=abc123. Mudah tapi tidak aman — terlihat di URL bar dan log server.

Hidden Field

Nilai tersembunyi di form HTML. Dikirim saat form disubmit. Berguna untuk multi-step form flow.

Komponen Utama Web

Core Web Components

Web Browser — Cara Kerjanya

User Interface

Address bar, tab, tombol navigasi, bookmark — bagian yang berinteraksi dengan pengguna

Browser Engine

Koordinasi antara UI dan rendering engine. Meneruskan instruksi user ke rendering

Rendering Engine

Parsing HTML & CSS, membangun DOM & CSSOM, menghitung layout, menampilkan halaman

JavaScript Engine

Menjalankan kode JS. Chrome: V8. Firefox: SpiderMonkey. Safari: JavaScriptCore

Networking

Mengelola HTTP/HTTPS request, DNS, caching, connection pool ke server

Data Storage

Cookie, localStorage, sessionStorage, IndexedDB — penyimpanan sisi klien

DevTools Browser — Alat Wajib Setiap Web Developer

Tekan F12 atau Klik Kanan → Inspect. DevTools adalah 'kaca pembesar' untuk web.

Elements

Inspeksi DOM tree dan computed CSS. Edit HTML/CSS langsung di browser untuk eksperimen cepat tanpa mengubah file.

Console

Error runtime JavaScript, log dari `console.log()`, dan bisa evaluasi ekspresi JS langsung. First stop saat debugging.

Network

Semua HTTP request yang dibuat halaman — status code, ukuran, waktu, header. Gunakan untuk debug API, CORS, dan redirect.

Sources

Lihat semua file sumber, pasang breakpoint, debug JavaScript step-by-step. Seperti IDE di dalam browser.

Application

Inspect dan edit cookie, localStorage, sessionStorage, IndexedDB. Lihat Service Worker dan Web App Manifest.

Lighthouse

Audit performa, aksesibilitas, SEO, dan PWA. Generate laporan dengan skor 0–100 dan rekomendasi perbaikan.

Panel Network: perhatikan kolom Waterfall — menunjukkan urutan dan ketergantungan loading resource. Gunakan Throttling untuk simulasi jaringan lambat (3G, 4G).

Lighthouse & Core Web Vitals — Ukuran Kualitas Web

Lighthouse — Audit Otomatis

Performa

Kecepatan loading halaman (LCP, FID, CLS)

Aksesibilitas

Bisa dipakai semua orang termasuk disabilitas

Best Practices

HTTPS, tidak ada deprecated API, security

SEO

Dapat diindeks mesin pencari dengan benar

PWA

Bisa diinstall, offline-capable, responsive

Core Web Vitals — Metrik Google

LCP

Largest Contentful Paint

Seberapa cepat konten utama terlihat

Good: ≤ 2.5 detik

CLS

Cumulative Layout Shift

Stabilitas visual — layout tidak loncat-loncat saat loading

Good: ≤ 0.1

INP

Interaction to Next Paint

Responsivitas interaksi: klik, ketik, tap

Good: ≤ 200 ms

Web Server — Dua Definisi

Web Server sebagai Software

Program yang menerima HTTP request dan mengirim HTTP response.

Tugas utama:

- Menerima & parsing HTTP request
- Mengembalikan file statis (HTML, CSS, JS, gambar)
- Meneruskan ke app server untuk konten dinamis
- Mengelola log, keamanan, SSL termination

Contoh populer:

- Apache HTTP Server
- Nginx (cepat, efisien untuk traffic tinggi)
- Microsoft IIS (untuk ekosistem Windows/.NET)
- Node.js built-in http module

Web Server sebagai Hardware

Komputer fisik atau virtual yang menjalankan software web server.

Karakteristik:

- Terhubung ke internet 24/7
- Memiliki IP publik atau terhubung via DNS
- Bisa berupa dedicated server, VPS, atau cloud

Provider cloud populer:

- AWS EC2, Google Cloud Compute, Azure
- DigitalOcean, Linode
- Vercel, Netlify (modern hosting)

Dalam percakapan sehari-hari, 'web server' biasanya merujuk ke keduanya sekaligus.

URI, URL, dan URN

URI — Uniform Resource Identifier

URL — Locator

URN — Name

Subset URI yang punya 'access mechanism' atau lokasi jaringan. Paling umum digunakan.

Contoh: <https://example.com/page?id=1#section>

Nama unik untuk resource, tanpa lokasi spesifik.

Contoh: urn:isbn:978-0-306-40615-7

Anatomi URL:

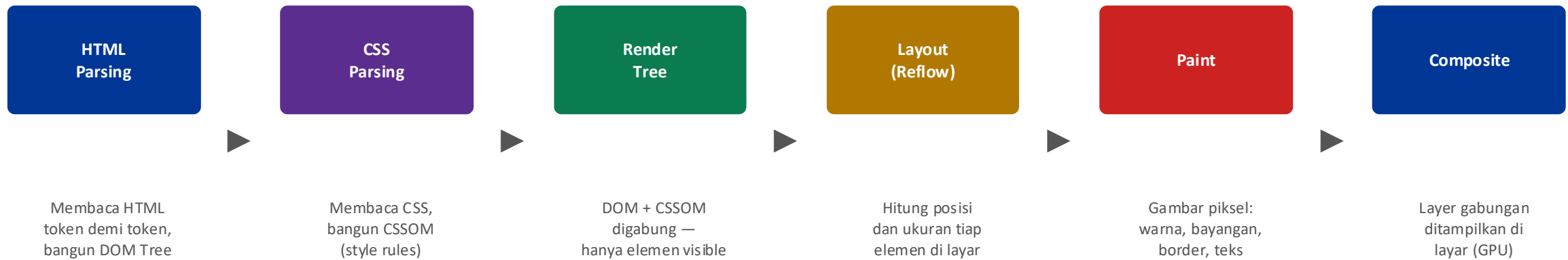
`https :// example.com :443 /path/page ?id=1&lang=id #section`

Rendering Browser & Teknologi Web

Browser Rendering & Web Technologies

Browser Rendering Pipeline

Setelah server mengirim HTML, inilah yang dilakukan browser:



Mengapa ini penting?

- Manipulasi DOM yang sering (misal dalam loop) memicu Reflow berulang → lambat
- CSS kompleks memperberat CSSOM → gunakan selector sederhana
- JavaScript blocking dapat menunda rendering → gunakan async/defer
- 'Critical Rendering Path' adalah konsep kunci optimasi performa web
- Di pertemuan HTML/CSS/JS, kita akan lihat dampak langsung dari rendering pipeline ini

Tiga Pilar Teknologi Front-End

HTML

HyperText Markup Language

Struktur & Konten

Mendefinisikan struktur dan semantik halaman web. Bukan bahasa pemrograman — tidak ada logika, hanya markup.

- Versi terbaru: HTML5 (2014)
- Tag semantik: <header>, <article>, <nav>
- Form, media (video, audio, gambar)
- Aksesibilitas via ARIA attributes

CSS

Cascading Style Sheets

Tampilan & Layout

Mengontrol bagaimana elemen HTML ditampilkan — warna, font, ukuran, posisi, animasi.

- Versi: CSS3 dengan banyak modul
- Flexbox & Grid untuk layout modern
- Media queries untuk responsive design
- Custom properties (CSS variables)

JS

JavaScript

Interaktivitas & Logika

Bahasa pemrograman satu-satunya yang native di browser. Menambah perilaku dinamis ke halaman.

- Standar: ECMAScript (ES6+ sejak 2015)
- Manipulasi DOM & event handling
- Async: fetch, Promise, async/await
- Juga berjalan di server via Node.js

Server-side vs Client-side Processing

Server-side Processing

Proses terjadi di server sebelum response dikirim ke browser.

- ✓ Keamanan data terjaga di server
- ✓ SEO lebih mudah (konten sudah ada)
- ✓ Tidak bergantung kemampuan browser
- X Setiap interaksi perlu round-trip ke server
- X Beban server lebih besar

Bahasa & framework:

PHP (Laravel), Python (Django/Flask),
Node.js (Express), Java (Spring),
Ruby (Rails), ASP.NET

Client-side Processing

Proses terjadi di browser pengguna setelah resource diterima.

- ✓ Responsif — tidak perlu ke server
- ✓ Mengurangi beban server
- ✓ Pengalaman pengguna lebih halus
- X Kode bisa dilihat & dimanipulasi user
- X Bergantung pada kemampuan browser

Teknologi:

HTML, CSS, JavaScript,
React / Vue / Angular,
WebAssembly

Aplikasi modern menggunakan keduanya: server-side untuk data & keamanan, client-side untuk interaktivitas & UX yang responsif.

Arsitektur Aplikasi Modern

Modern Web App Architecture

MPA vs SPA vs SSR — Tiga Pendekatan Rendering

MPA

Multi-Page App

Setiap navigasi memuat halaman baru dari server secara penuh.

- ✓ SEO sangat baik (konten di HTML)
- ✓ Sederhana, tanpa JS untuk navigasi
- ✗ Full page reload di setiap klik
- ✗ Lebih lambat untuk interaksi intens

Contoh: Wikipedia, situs berita, e-gov

SPA

Single-Page App

Satu halaman HTML, navigasi dikelola JavaScript — hanya data yang diperbarui.

- ✓ Interaksi sangat cepat setelah load
- ✓ Pengalaman mirip aplikasi native
- ✗ SEO lebih sulit (butuh konfigurasi)
- ✗ Bundle JS besar → load awal lambat

Contoh: Gmail, Trello, Figma (React/Vue/Angular)

SSR

Server-Side Rendering

HTML di-render di server tiap request, tapi hydrated oleh JS di browser.

- ✓ SEO baik (HTML lengkap dari server)
- ✓ Load awal lebih cepat (First Paint)
- ✗ Server harus render setiap request
- ✗ Lebih kompleks dari MPA murni

Contoh: Next.js, Nuxt.js, SvelteKit

CDN — Content Delivery Network

Masalah: Server tunggal di satu lokasi → pengguna di lokasi jauh mendapat respons yang lambat.

Solusi: CDN mendistribusikan salinan konten ke banyak server (PoP — Point of Presence) di seluruh dunia.

Performa

Pengguna dilayani dari server terdekat → latensi rendah, load lebih cepat

Availability

Jika satu server down, server lain mengambil alih → high availability

Keamanan

CDN dapat mitigasi DDoS dan menyediakan SSL termination terpusat

Apa yang di-cache CDN: Gambar, CSS, JavaScript, video, font, dokumen statis

Provider populer: Cloudflare, AWS CloudFront, Fastly, jsDelivr

Web framework modern (Next.js, Vercel, Netlify) sudah menyediakan CDN otomatis untuk aset statis. Mahasiswa tidak perlu setup CDN sendiri untuk proyek kecil.

MVC Pattern & Web Framework

MVC & Web Frameworks

Three-Tier Architecture & Web Framework

Three-Tier Architecture

Presentation Tier

Browser, UI — apa yang dilihat user

Logic Tier

Application server — business logic, rules

Data Tier

Database — penyimpanan data persisten

Web Framework

Framework menyediakan struktur dan tools untuk membangun aplikasi web:

- Routing URL ke handler/controller
- Templating engine (Blade, Jinja2, EJS)
- ORM untuk akses database
- Authentication & middleware
- Validasi input & security defaults

Contoh framework populer:

PHP: Laravel

Python: Django

Node.js: Express

Java: Spring

MVC — Model-View-Controller

Pola arsitektur software yang memisahkan aplikasi menjadi tiga komponen berdasarkan peran.

Model

Mengelola data dan logika bisnis aplikasi.

- Merepresentasikan struktur data (tabel, entitas)
- Berinteraksi dengan database
- Memvalidasi data sebelum disimpan
- Memberitahu View ketika data berubah

Contoh: class User, class Product, DAO/Repository

View

Menampilkan data dari Model ke pengguna.

- Template HTML yang diisi data dari Model
- Tidak mengandung logika bisnis
- Mengirim input user ke Controller
- Diupdate saat Model berubah

Contoh: user.html, product_list.blade.php

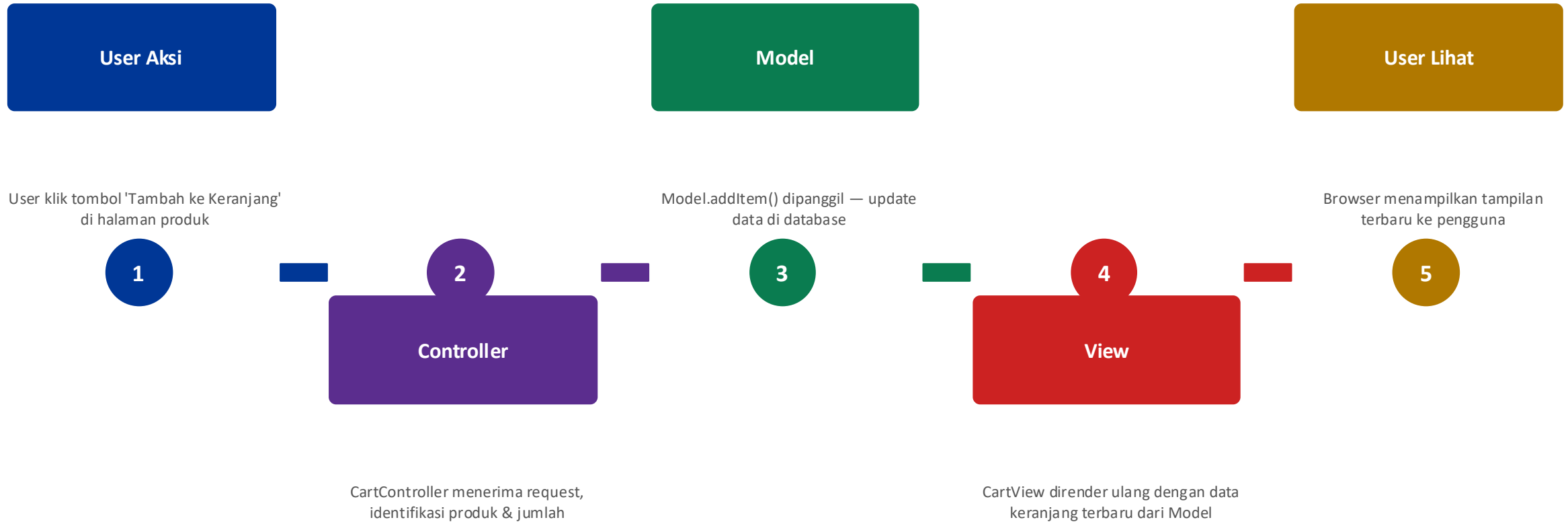
Controller

Mediator antara Model dan View.

- Menerima HTTP request dari browser
- Memanggil Model yang sesuai
- Memutuskan View mana yang ditampilkan
- Mengarahkan alur data aplikasi

Contoh: UserController, ProductController

Alur Kerja MVC — Contoh Shopping Cart



MVC — Keunggulan dan Keterbatasan

Keunggulan (Advantages)

- ✓ Separation of concerns yang jelas — setiap komponen punya satu tanggung jawab
- ✓ Mendukung parallel development — tim front-end & back-end bisa kerja bersamaan
- ✓ Mudah di-test — Model bisa di-test tanpa View atau Controller
- ✓ Reusable — satu Model bisa digunakan oleh banyak View
- ✓ Mudah dimaintain dan dikembangkan ke depan

Keterbatasan (Disadvantages)

- ✗ Menambah kompleksitas untuk aplikasi kecil/ sederhana
- ✗ Learning curve — butuh waktu untuk memahami pola ini
- ✗ Data access di View bisa tidak efisien jika desain kurang tepat
- ✗ Membutuhkan beberapa programmer untuk parallel development
- ✗ Perlu pemahaman banyak teknologi sekaligus

Referensi & Sumber Belajar

Dokumentasi & Bacaan Utama

MDN Web Docs

`developer.mozilla.org/en-US/docs/Learn`

Dokumentasi web paling otoritatif. Panduan How the Web Works.

web.dev

`web.dev/articles/howbrowserswork`

Artikel lengkap cara kerja browser dari Google Engineers.

HTTP/2 Explained

`http2-explained.haxx.se`

E-book gratis tentang HTTP/2 oleh Daniel Stenberg (penulis curl).

History of the Web

`thehistoryoftheweb.com`

Timeline interaktif sejarah web yang sangat lengkap.

Video untuk Flipped Learning

Traversy Media

How The Web Works (Full Course)

Fireship

HTTP Crash Course & Cheat Sheet

Fireship

DNS in 100 Seconds

CS50 Web

Harvard, `cs50.harvard.edu/web`

Kursus web terlengkap, gratis, dengan video lecture